

# Kimi K3 in vLLM

How we build it to achieve 460 tok/s

Jiangyun Zhu@Inferact    Yongye Zhu@Inferact

# Kimi K3 Architecture Overview

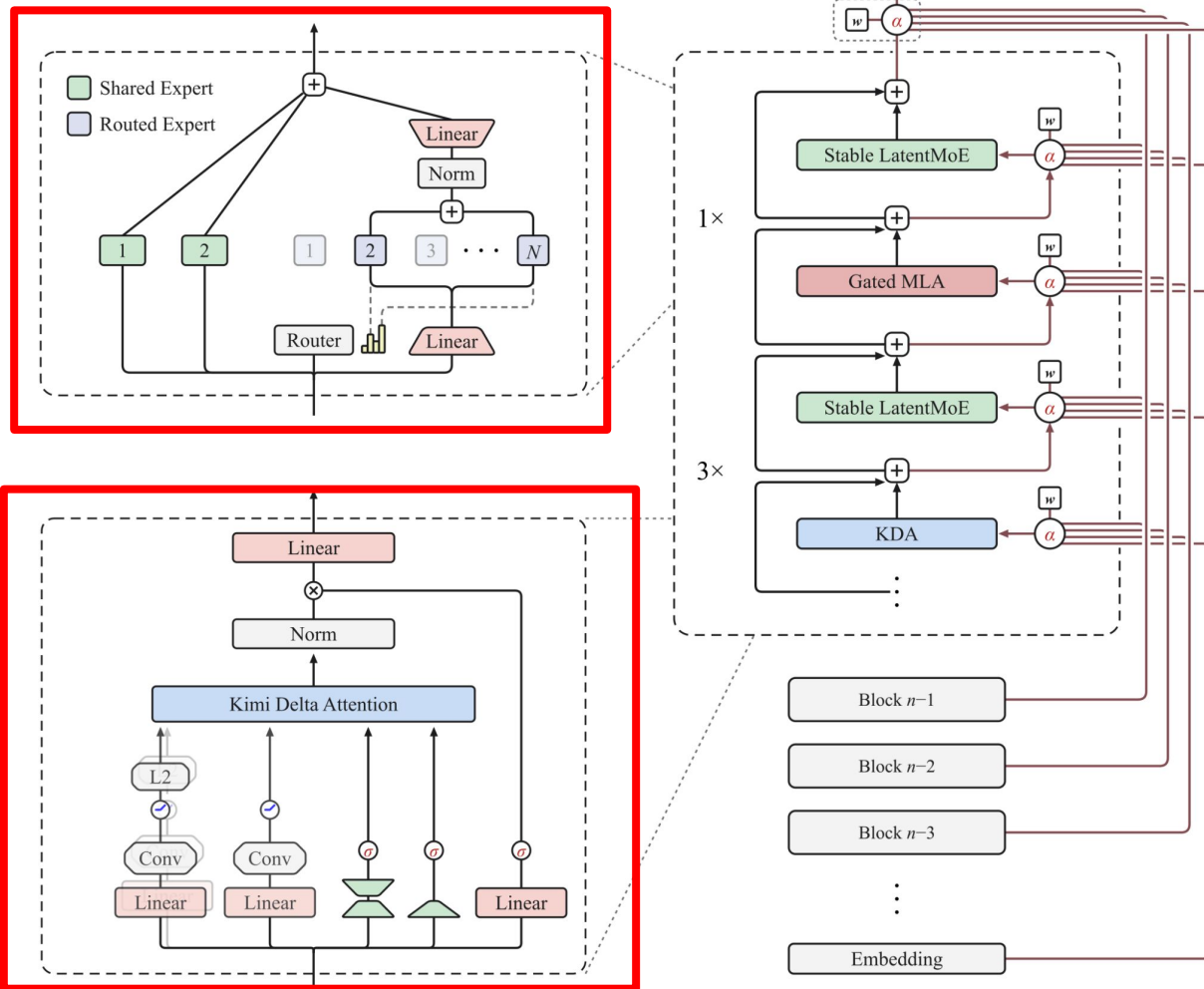
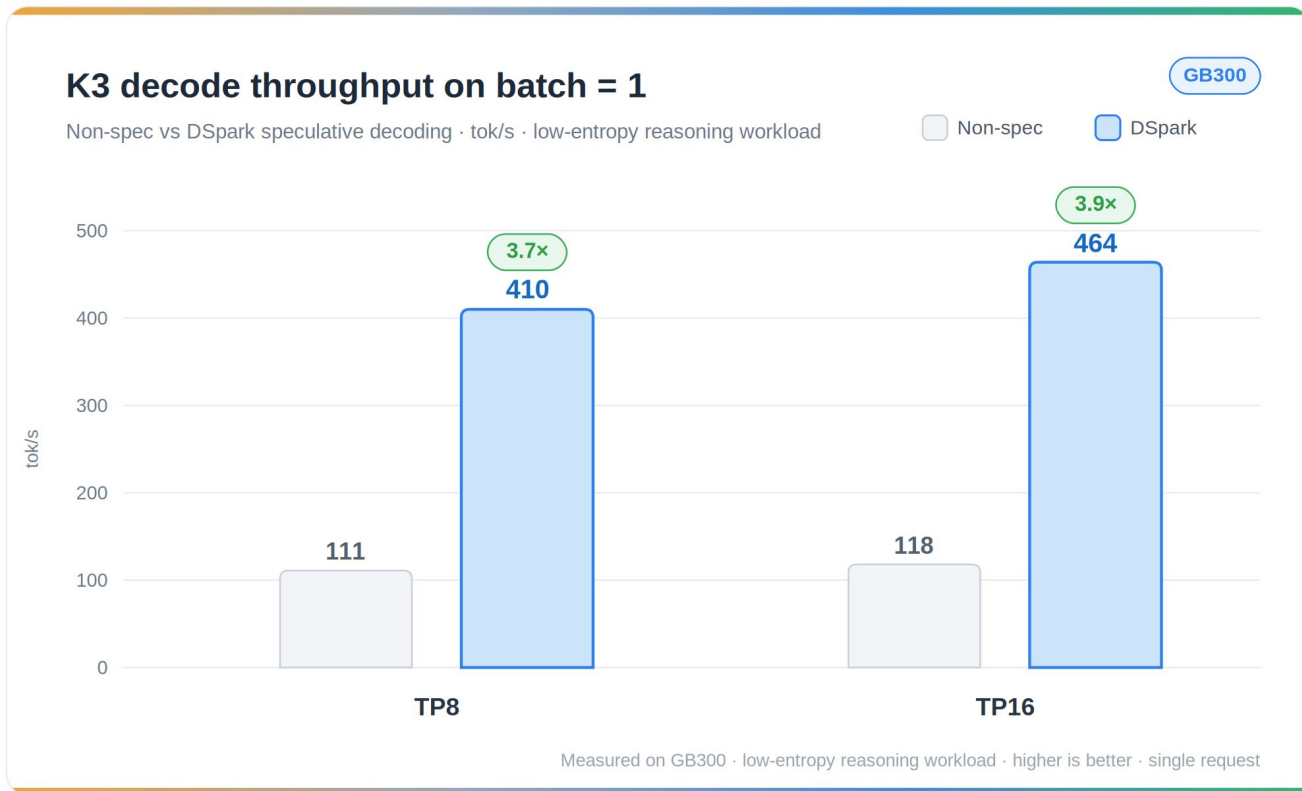


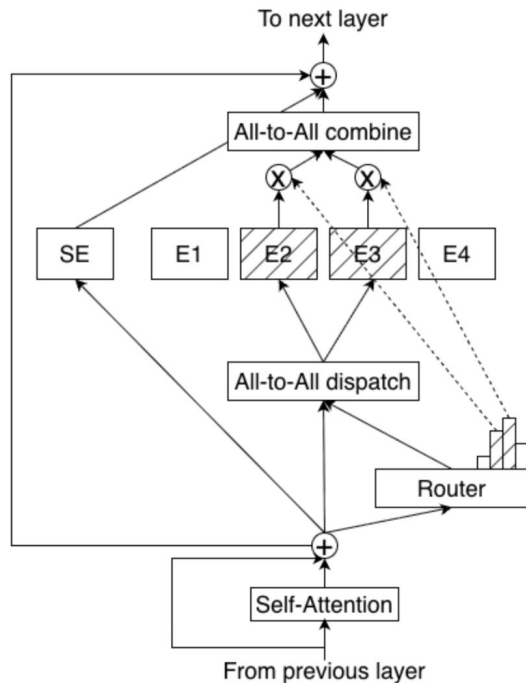
Table 1: Architectural comparison between Kimi K2 and Kimi K3.

|                                 | <b>Kimi K2</b> | <b>Kimi K3</b>       | <b><math>\Delta</math></b> |
|---------------------------------|----------------|----------------------|----------------------------|
| Architecture                    | MoE            | MoE                  | –                          |
| #Layers                         | 61             | 93                   | $\uparrow 52\%$            |
| Total Parameters                | 1.04T          | 2.78T                | $\uparrow 167\%$           |
| Activated Parameters            | 32.6B          | 104.2B               | $\uparrow 220\%$           |
| Hidden Dimension                | 7,168          | 7,168                | =                          |
| Latent MoE Dimension            | –              | 3584 (0.5 $\times$ ) | –                          |
| MoE Hidden Dimension per Expert | 2,048          | 3,072                | $\uparrow 50\%$            |
| Routed Experts                  | 384            | 896                  | $\uparrow 133\%$           |
| Experts Active per Token        | 8              | 16                   | $\uparrow 100\%$           |
| Shared Experts                  | 1              | 2                    | $\uparrow 100\%$           |
| Attention Heads                 | 64             | 96                   | $\uparrow 50\%$            |
| Number of Dense Layers          | 1              | 1                    | =                          |
| Vocabulary Size                 | 160K           | 160K                 | =                          |
| Training Context Length         | 128K           | 1M                   | 8 $\times$                 |
| Attention Mechanism             | MLA            | Hybrid KDA–MLA       | –                          |
| Activation Function             | SwiGLU         | SITU-GLU             | –                          |
| Attention-Layer Composition     | 61 MLA         | 69 KDA + 24 MLA      | –                          |
| Number of MTP Layers            | 1 layer        | 1 layer              | =                          |
| Total Parameters of ViT         | –              | 401M                 | –                          |
| #ViT Layers                     | –              | 27 layers            | –                          |
| Patch Size of ViT               | –              | 14                   | –                          |
| #Attention Heads of ViT         | –              | 12                   | –                          |

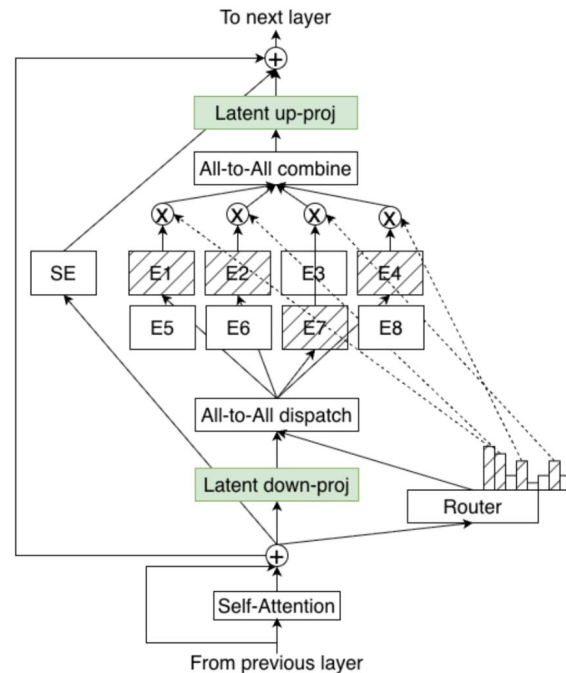
# Performance



# LatentMoE: reduce communication



(a) Standard MoE architecture.



(b) LatentMoE architecture.

# Kimi Delta Attention

## 3 Kimi Delta Attention: Improving Delta Rule with Fine-grained Gating

We propose Kimi Delta Attention (KDA), a new gated linear attention variant that refines GDN's scalar decay by introducing a fine-grained diagonalized gate  $\text{Diag}(\alpha_t)$  that enables fine-grained control over memory decay and positional awareness (as discussed in §6.1). We begin by introducing the chunkwise parallelization of KDA, showing how a series of rank-1 matrix transformations can be compressed into a dense representation while maintaining stability under diagonal gating. We then highlight the efficiency gains of KDA over the standard DPLR (*Diagonal-Plus-Low-Rank*) formulation [30, 71].

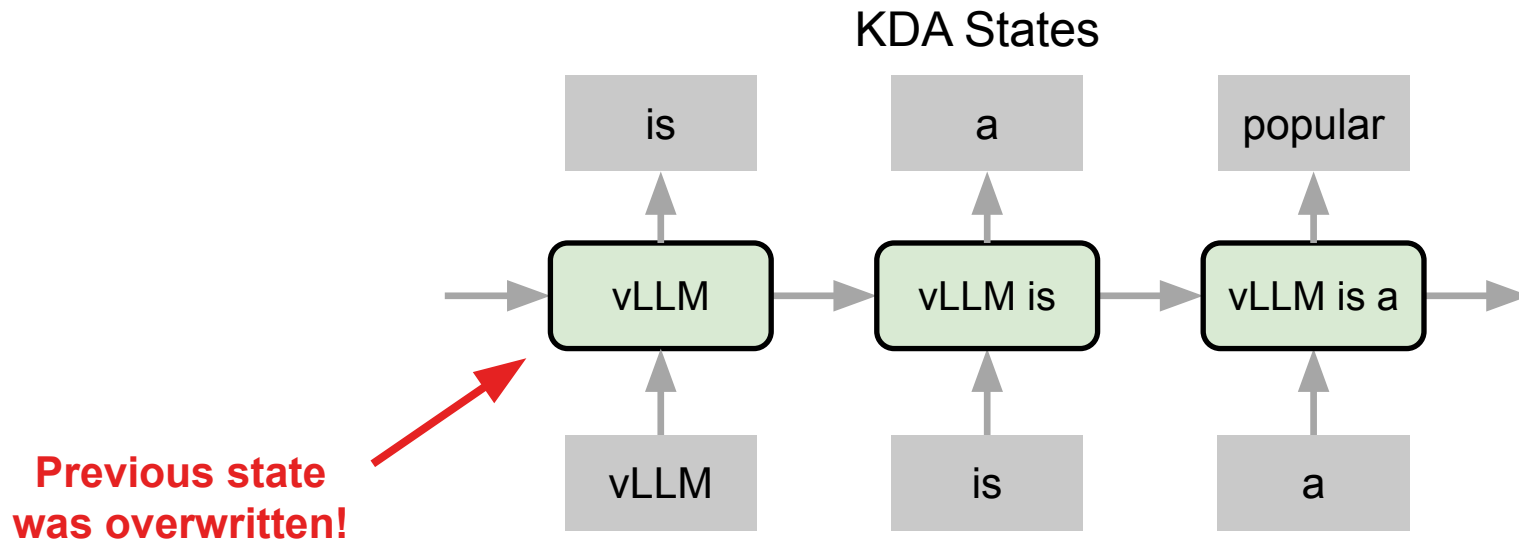
$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \in \mathbb{R}^{d_v} \quad (1)$$

The diagram illustrates the matrix operations in the KDA equation. It shows a sequence of matrix multiplications and additions using colored blocks to represent matrices and vectors. The first part shows  $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1}$  where  $\mathbf{I}$  is a diagonal matrix,  $\mathbf{k}_t$  is a column vector,  $\mathbf{k}_t^\top$  is a row vector, and  $\mathbf{S}_{t-1}$  is a matrix. The second part shows  $\beta_t \mathbf{k}_t \mathbf{v}_t^\top$  where  $\mathbf{k}_t$  is a column vector and  $\mathbf{v}_t^\top$  is a row vector. The final part shows the output  $\mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$  where  $\mathbf{S}_t$  is a matrix and  $\mathbf{q}_t$  is a column vector.

<https://zhiyuan1i.github.io/posts/kda-mathematics>

# Challenging

- states can't be rolled back to represent a prefix



**SSM's modeling win complicates their systems win!**

# Challenging

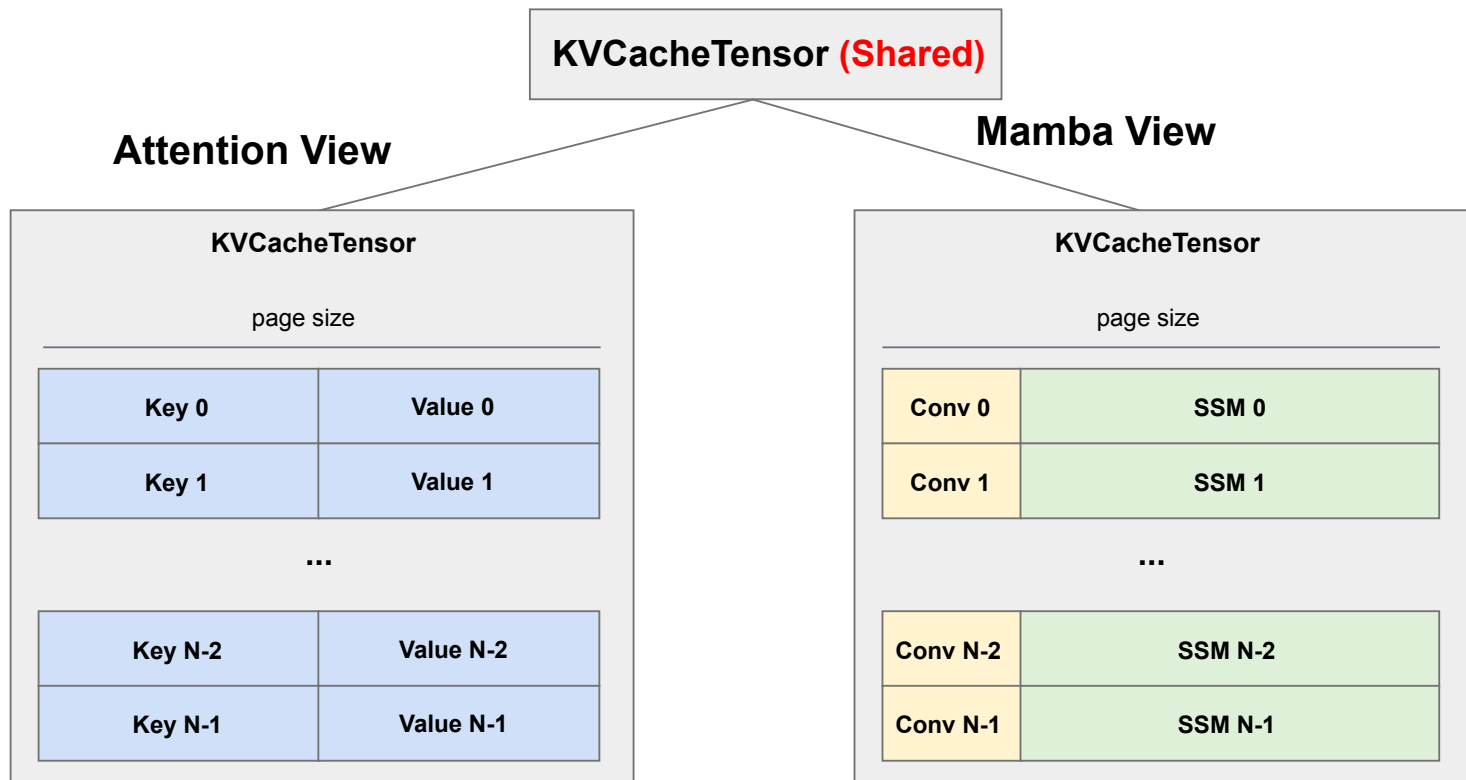
- This breaks the assumption of vLLM: the kv cache grows continuously and is append-only.
- How to do:
  - **memory management**
  - **prefix caching**

## KV Cache

|      |    |   |         |
|------|----|---|---------|
| vLLM | is | a | popular |
|------|----|---|---------|



# How to manage: Hybrid Memory Allocator



# Hybrid Memory Allocator

- Split the memory into pages whose size is the **Least Common Multiple (LCM)** of all possible allocation sizes
- For each type, further split one LCM page into multiple small pages of specific size with **no fragmentation**

**Data Type 1** (full attn)

**Size X = 1 KB**

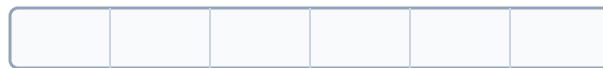
**Data Type 2** (sliding window)

**Size Y = 2 KB**

**Data Type 3** (mamba)

**Size Z = 3 KB**

**LCM Page** (Size =  $\text{LCM}(X, Y, Z) = 6 \text{ KB}$ )



LCM Page



**6 × 1 KB**



**3 × 2 KB**



**2 × 3 KB**

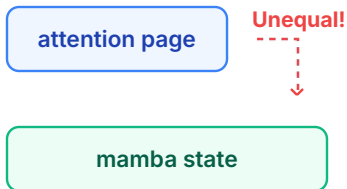
# Page Size Alignment

*Aligning page sizes between Attention and Mamba models to enable hybrid KV caching*

01

## Calculate Original Sizes

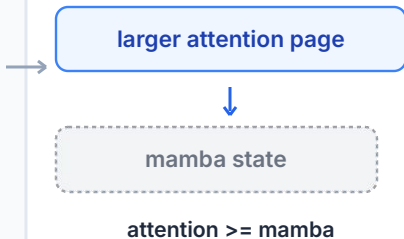
Calculate original size of both page types, which are usually unequal



02

## Adjust block\_size

Make an attention page large enough to accommodate Mamba



03

## Padding Mamba

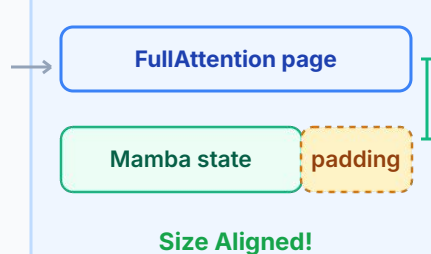
If still unequal, pad the Mamba page to match attention size



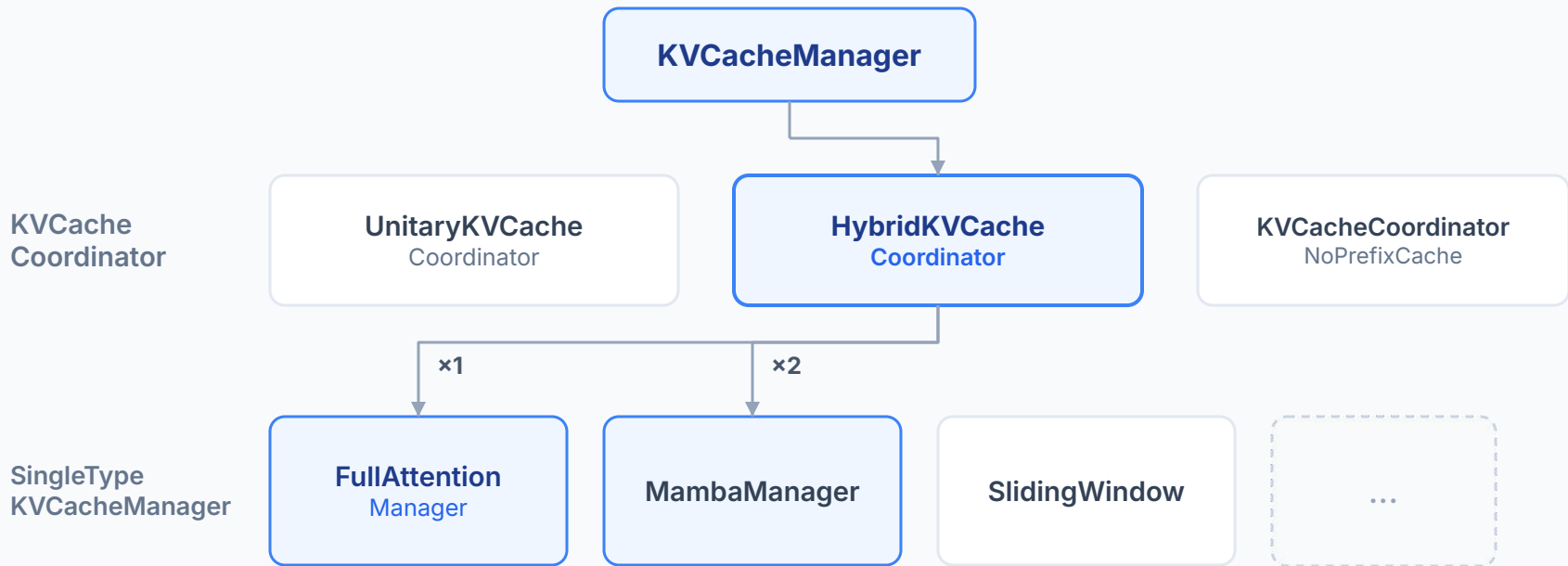
04

## Aligned Result

Ensure identical page\_size\_bytes for each KV cache group



# Hybrid KV Cache Manager



# Block-aligned Scheduling

*Prefix caching for Mamba / linear-attention models in vLLM*

## 1 Two prerequisites

States must be bound to a **Block Hash** to be reused on a cache hit

Linear attention only emits the **final state** — it cannot be rolled back per token

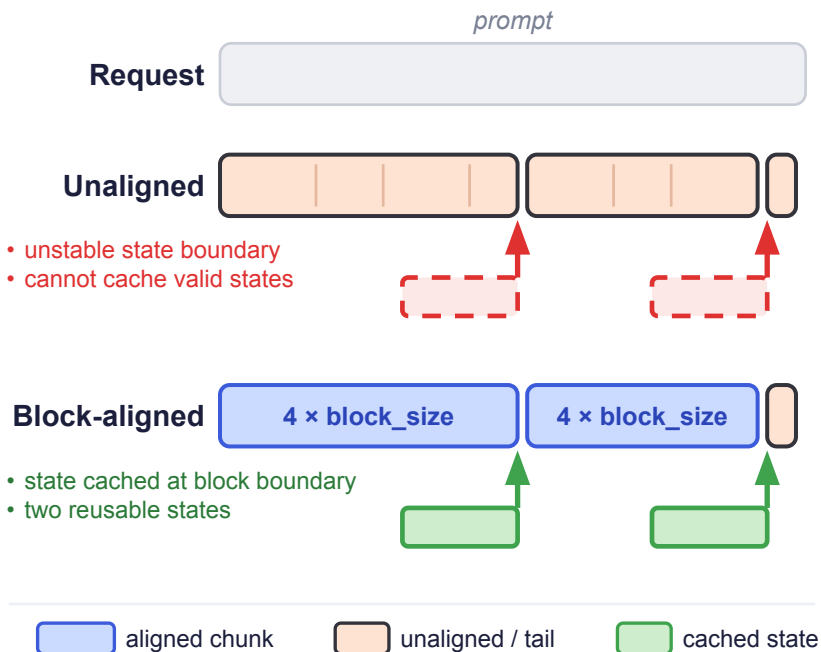
## 2 Solution

Align every prefill chunk to a **multiple of block\_size**, so each state lands on a block boundary — only the **last chunk** is unconstrained

## 3 Key advantages

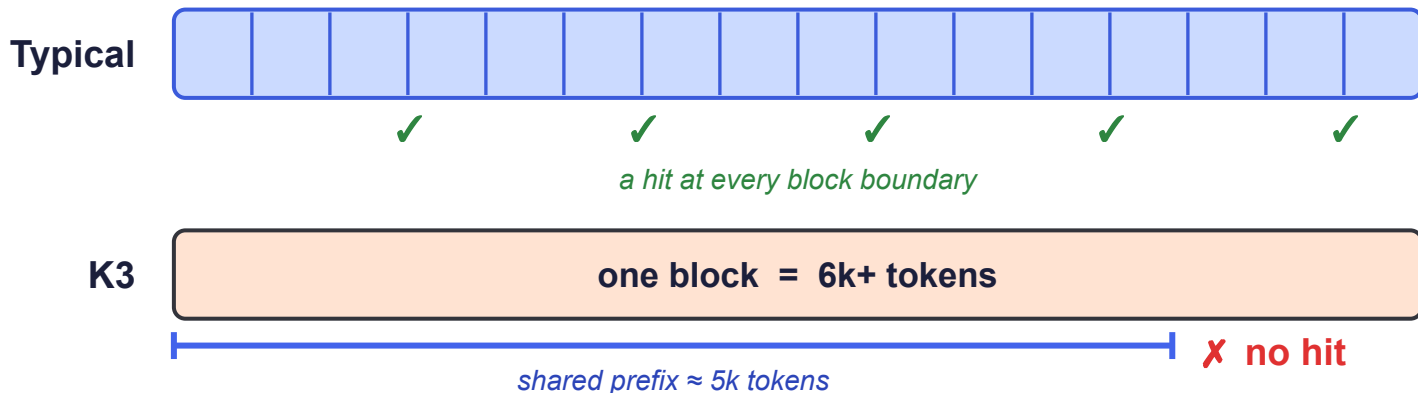
Scheduler-driven — **zero model-layer changes**

Balances **GPU memory** footprint against **cache-hit rate**

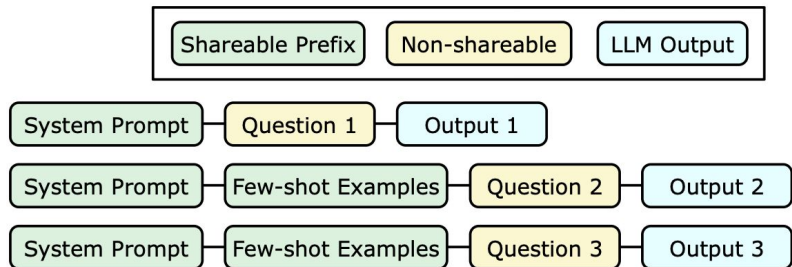


# Weakness of existing design

- We need to align the page sizes across different KV cache groups.
- As a result, if one cache is much larger than another, the smaller cache may require a very large block size to match it.
- K3: KDA state is about 600x larger than MLA cache, the block size of K3 is 6k+ in DP case, this makes prefix cache very inefficient

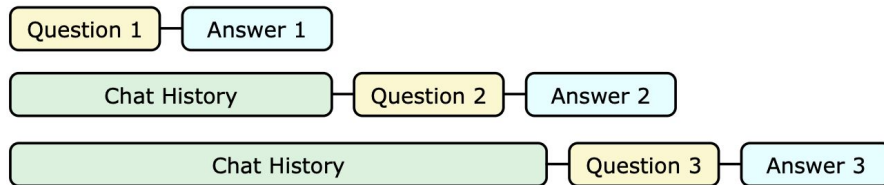


# What's the most valuable cache



(a) System prompt and few-shot prompting

System Prompt



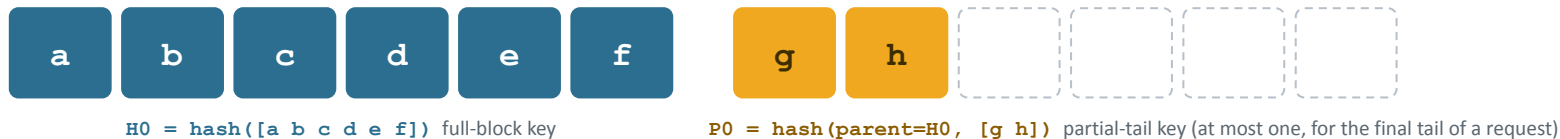
(b) Multi-turn conversation (e.g., ChatGPT)

The boundary of each turn

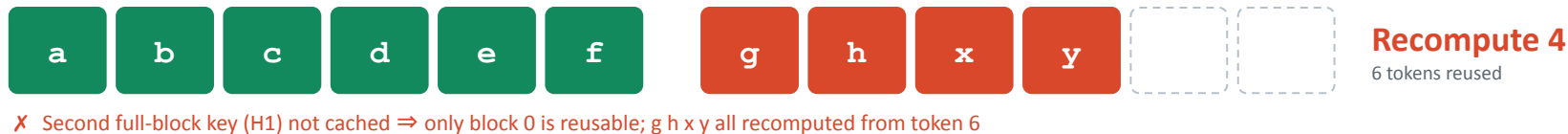
# Partial Cache Hit: Decouple Cache-Hit Granularity from Block Size

vLLM RFC #45702 (proposed by Moonshot) — introduce `hash_block_size` to hit a partial cache block **Example: `block_size = 6`, `hash_block_size = 2`**

**Request A** prompt = a b c d e f g h (written to cache; registers both kinds of hash keys)

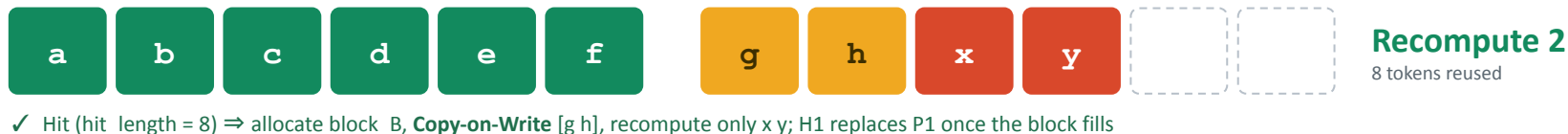


**Request B (before)** prompt = a b c d e f g h x y — full-block (`block_size`) matching only



✗ Second full-block key (`H1`) not cached  $\Rightarrow$  only block 0 is reusable; g h x y all recomputed from token 6

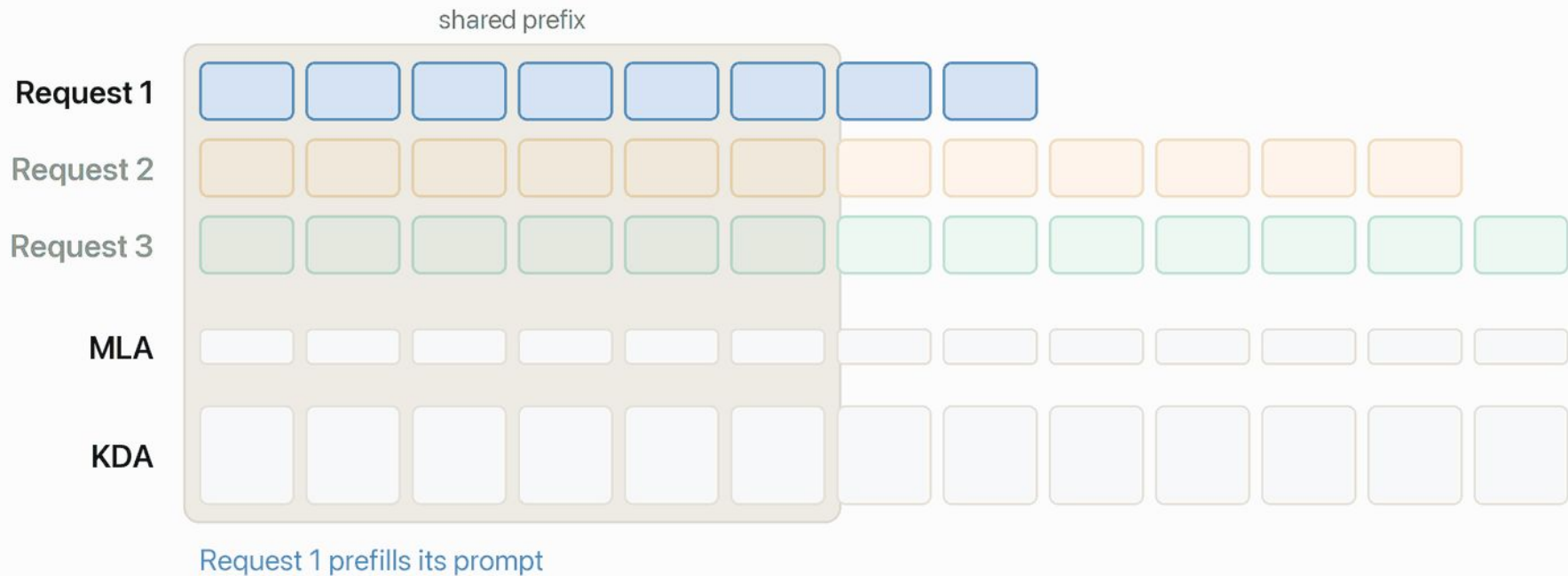
**Request B (proposed)** after the first full-block miss, probe partial-tail keys inside that block



✓ Hit (`hit_length = 8`)  $\Rightarrow$  allocate block\_B, **Copy-on-Write** [g h], recompute only x y; `H1` replaces `P1` once the block fills



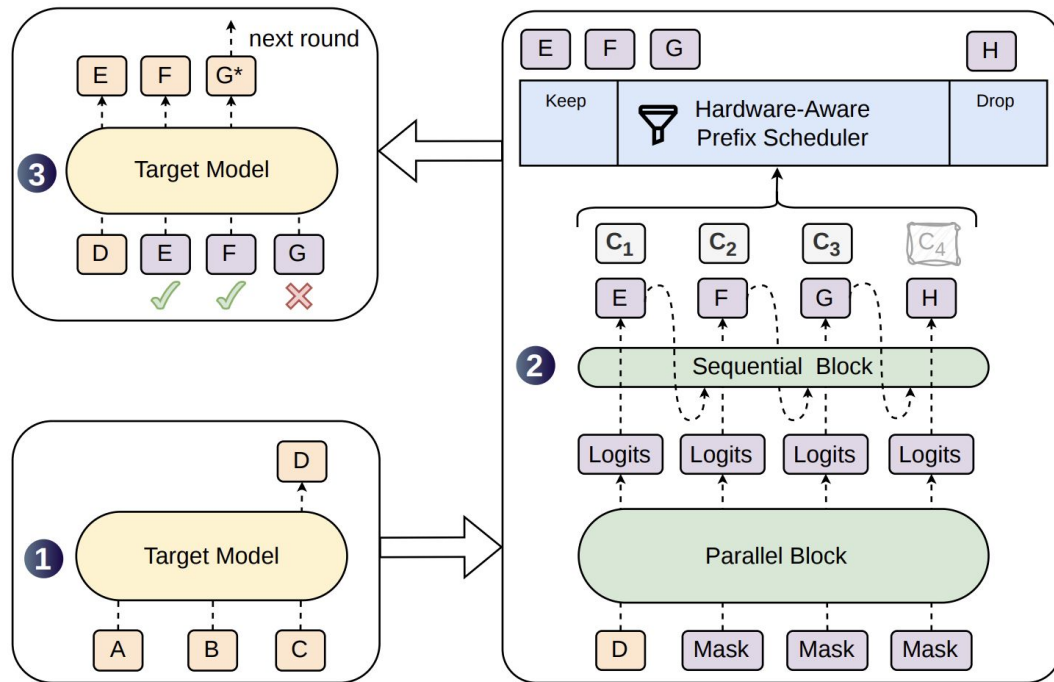
# Marconi-style selective retention



## Others

- Tensor reuse may lead to NaN, need to zero block before reusing
- Zeroing and RDMA write has data race when using PD

# Dspark: Inferact/Kimi-K3-DSpark



# Low Latency Optimization

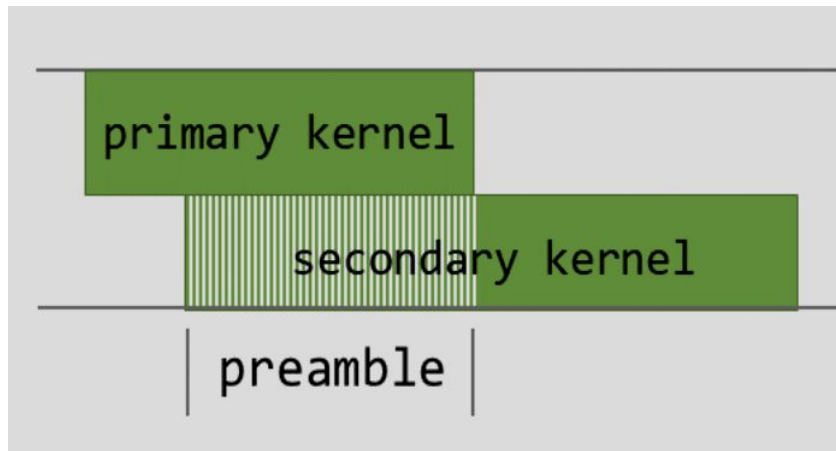
- Aggressively Using Programmatic Dependent Launch (PDL)
- Tail MoE optimizations

# Programmatic Dependent Launch

- Before PDL, two kernels are launched sequentially
- After PDL, two kernels can be partial overlapped.



Before



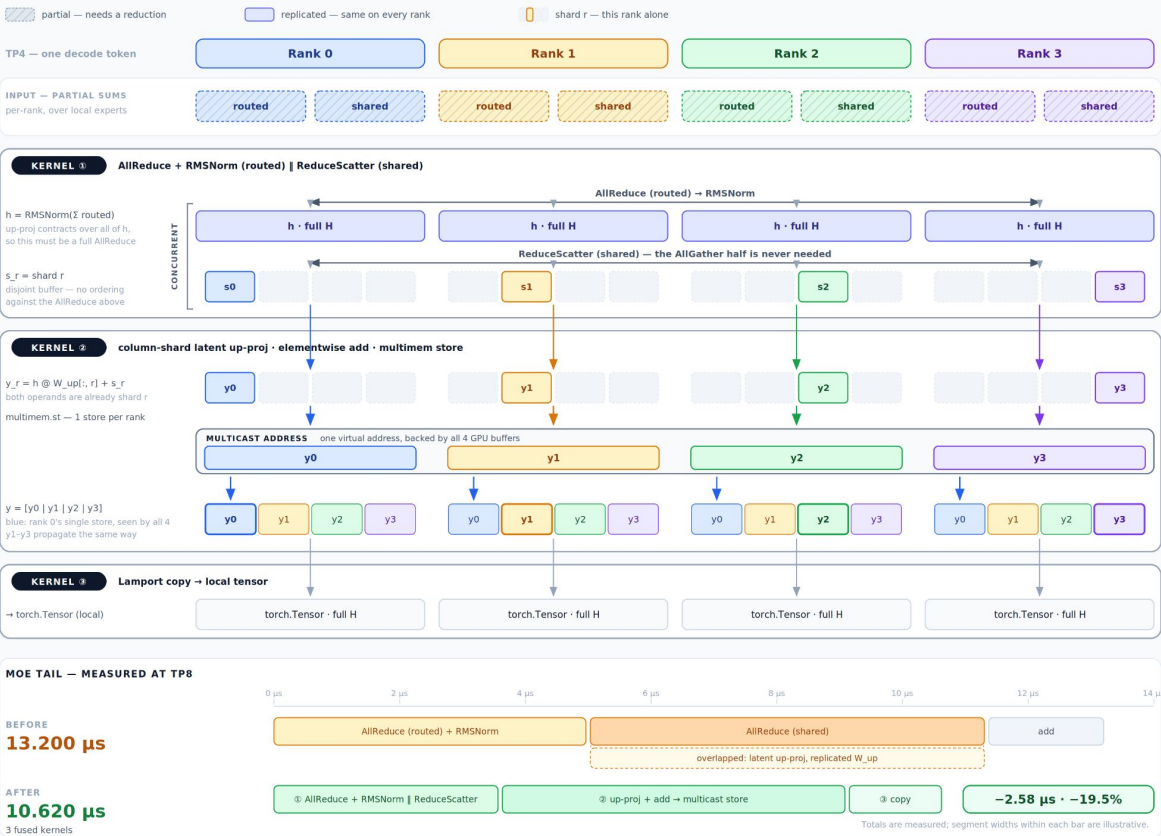
After

# Latent MoE Tail

- Fuse reduction for routed experts and shared experts
- Shard the latent up projection column wise so that we don't replicate the compute.

## Sharding the MoE Tail — Why ReduceScatter Replaces an AllReduce

Follow one decode token through 3 fused kernels at TP4 · Kimi K3 decode, measured at TP8 (2 nodes × 4 GB300)



# Reference

- <https://vllm.ai/blog/2026-07-27-k3>
- <https://recipes.vllm.ai/moonshotai/Kimi-K3>
- <https://mlsys.org/virtual/2025/poster/3260>
- <https://www.kimi.com/blog/kimi-k3>
- <https://arxiv.org/abs/2601.18089>
- <https://github.com/vllm-project/vllm/pull/50000>
- <https://zhiyuan1i.github.io/posts/kda-mathematics>
- <https://github.com/MoonshotAI/Kimi-Linear>
- <https://github.com/MoonshotAI/Kimi-K3>
- <https://huggingface.co/Inferact/Kimi-K3-DSpark>